Adaptive Additive Quantization for Vector Search

Astitwa Saxsena (2025MCS3005)
Abhishek Gupta(2025MCS2963)
Nizaul Rahman (2025MCS2099)
Department of Computer Science and Engineering
IITD

April 22, 2026

Abstract

Modern applications like search engines and recommendation systems use vector similarity search to find similar items. However, checking all data points is very slow, so systems use Approximate Nearest Neighbor (ANN) methods, which are faster but less accurate. In this project, we design a vector search system that balances speed and accuracy. We use three main ideas: (1) IVF indexing to reduce the search space, (2) Additive Quantization (AQ) to compress vectors and speed up comparison, and (3) an adaptive reranking method that improves results only when needed. The key idea is simple: if the system is confident about the result, it returns it quickly; if the result is uncertain, it spends extra time to improve accuracy using exact distance calculations. Our experiments on datasets like MNIST, SIFT, and GloVe show that this approach gives much better accuracy than standard methods. This shows that adapting computation based on query difficulty is an effective way to build efficient and accurate vector search systems.

1 Introduction

Modern applications such as search engines, recommendation systems, and large language models rely heavily on vector embeddings to represent data. These embeddings are high-dimensional, and finding similar vectors efficiently is a key challenge.

A straightforward approach is to compare a query with every vector in the database. However, exact search is computationally expensive and does not scale to large datasets. To address this, systems use Approximate Nearest Neighbor (ANN) methods, which improve speed but often reduce accuracy.

In this work, we propose a vector search system that balances efficiency and accuracy by adapting computation based on query difficulty. Instead of treating all queries equally, the system dynamically decides how much computation is needed. For easy queries with a clear best match, results are returned quickly. For harder queries with ambiguous candidates, additional refinement is performed using exact distance computation.

Our approach combines three key ideas.

- First, we use Inverted File (IVF) indexing to reduce the search space.
- Second, we apply Additive Quantization (AQ) to compress vectors and enable fast similarity computation.
- Finally, we introduce an adaptive reranking step that selectively refines uncertain results.

The system is implemented using FAISS [1], which provides efficient support for large-scale vector indexing and search.

This design allows the system to achieve high accuracy without always incurring the cost of expensive computation, similar to how a database query optimizer selectively applies more complex operations only when necessary.

The implementation is publicly available 9, enabling reproducibility of all experiments.

2 System Architecture

Our system follows a **three-layer pipeline**, where each layer progressively refines the search results.

Given a query vector, the system first reduces the search space using an index structure, then performs efficient approximate similarity computation, and finally applies selective refinement to improve accuracy when necessary.

2.1 Layer 1: IVF Indexing (Query Pruning)

In the first stage, we use an Inverted File (IVF) index to partition the dataset into clusters (inverted lists). Each vector is assigned to a cluster based on its proximity to a centroid.

At query time, instead of searching the entire dataset, the system identifies the nearest clusters and restricts the search to those regions. The number of clusters explored is controlled by the parameter *nprobe*.

A smaller *nprobe* results in faster queries but may miss relevant neighbors, while a larger *nprobe* improves accuracy at the cost of increased computation. The parameter n_{list} controls the number of clusters in the index: larger n_{list} leads to finer partitioning and fewer vectors per cluster, improving pruning efficiency but increasing index overhead. Overall, this step reduces the search complexity from $O(N)$ to approximately $O(N/n_{list})$.

2.2 Layer 2: Additive Quantization (Storage Compression)

In the second stage, we apply Additive Quantization (AQ) to compress vectors and enable efficient similarity computation. Each vector is approximated as a sum of M codewords:

$$x \approx \sum_{i=1}^{M} C_i$$

This representation allows compact storage and faster distance estimation.

Compared to Product Quantization (PQ), which splits vectors into independent subspaces, AQ provides a more expressive representation by modeling dependencies across dimensions, at the cost of higher computational complexity. This leads to lower reconstruction error and improved retrieval accuracy.

2.3 Layer 3: Adaptive Reranking (Query Optimization)

The final stage introduces an adaptive reranking mechanism, which is the key contribution of our system.

After retrieving approximate nearest neighbors, the system estimates the confidence of the results using a distance-based metric. Specifically, we define a confidence gap:

$$\Delta = d_k - d_1$$

where d_1 is the distance to the closest candidate and d_k is the distance to the k-th candidate.

The confidence gap measures how well the top result is separated from the rest of the candidates. A large gap indicates a confident query with a clearly better match, while a small gap suggests that multiple candidates are similarly close, making the result ambiguous. In such cases, the system performs additional refinement to improve accuracy. A large gap is considered better because it means the best match is clearly separated from the rest.

Based on this observation, the system dynamically adjusts its behavior. For confident queries, results are returned immediately. For ambiguous queries, the system retrieves a larger candidate set and performs reranking using exact L2 distance on the original vectors.

This selective refinement strategy ensures that additional computation is applied only when necessary, improving accuracy without significantly increasing average latency.

The system is implemented using FAISS [1], which provides efficient IVF-based approximate nearest neighbor search. Product Quantization (PQ) [2] and Additive Quantization (AQ) [4] are used for vector compression and indexing.

The reranking step uses exact L2 distance computation on the original vectors for a small set of shortlisted candidates. While Additive Quantization (AQ) is used for efficient indexing and candidate retrieval, the final ranking step intentionally switches to exact distance evaluation to eliminate quantization error.

3 Limitations of Baselines

PQ:

- High compression error
- Poor accuracy in high dimensions

OPQ:

- Slight improvement over PQ
- Still independent subspace assumption

Key Insight: Neither PQ nor OPQ adapts dynamically to query difficulty.

4 Limitations of Baselines

Product Quantization (PQ): [2]

- PQ introduces high compression error due to coarse vector partitioning.
- It performs poorly in high-dimensional spaces where subvector independence breaks down.

Optimized Product Quantization (OPQ):

- OPQ improves over PQ by learning an optimal rotation of the input space.
- However, it still relies on the assumption of independent subspaces after rotation, which limits expressiveness.

Key Insight: Neither PQ nor OPQ adapts its computation based on query difficulty. Both methods apply a fixed approximation strategy for all queries, regardless of whether a query is easy or ambiguous. This lack of adaptivity motivates our proposed approach.

5 Complexity Analysis

Baseline PQ/OPQ:

$$O(nprobe \cdot M)$$

where *nprobe* controls the number of clusters searched and M is the number of sub-quantizers.

Additive Quantization (AQ): [4] AQ has higher computational overhead due to multi-codebook distance computation:

$$O(nprobe \cdot M \cdot k)$$

where k is small constant from lookup operations, with a larger constant factor compared to PQ/OPQ due to residual encoding and codebook interactions.

Adaptive Reranking: Worst case:

$$O(C \cdot d)$$

where C is the candidate pool size and d is the dimensionality. However, this step is triggered only for ambiguous queries.

More generally, expected cost is:

$$O(p \cdot C \cdot d)$$

where p is the probability of triggering reranking.

Interpretation:

- PQ/OPQ are efficient due to lightweight approximate distance computation.
- AQ improves accuracy but introduces higher constant computational overhead.
- Adaptive reranking increases cost only for difficult queries.

6 Hyperparameter Study of Adaptive AQ

6.1 Goal

The purpose of this experiment is to understand how different parameters affect:

- Accuracy (Recall)
- Speed (Latency)

We focus on four key parameters:

- M: number of codebooks (controls compression quality)
- *nlist*: number of clusters (index granularity)
- *nprobe*: number of clusters searched (search depth)
- Δ: confidence threshold (controls reranking)

6.2 Method (What the Code Does)

For each combination of parameters, we run the following steps:

1. Build a FAISS IVF + Quantization index using M and *nlist*
2. For each query:
   (a) Perform an initial search to get top candidates
   (b) we compute exact L2 distances on original vectors for reranking candidates
   (c) Sort them and compute the **confidence gap**:
   $$\text{gap} = d_k - d_1$$
   (d) If the gap is small ($< \Delta$):
   - Treat query as **hard**
   - Perform a larger search (reranking)

   (e) Otherwise:
   - Treat query as **easy**
   - Use current results directly
3. Measure:
   - Recall (accuracy): How many of the correct nearest neighbors the system successfully retrieves. Higher recall means better search quality.

- Latency per query: The average time taken to process and return results for one query. Lower latency means faster response.
- Trigger rate (% of hard queries): The percentage of queries that are considered "hard" and need extra computation (reranking). A higher trigger rate means more queries require additional processing.

6.3 Observations MNIST

- **Effect of Δ:** Small Δ (0.02) $\rightarrow$ very few reruns ($\sim$1%) $\rightarrow$ fast.
  Large Δ (0.10) $\rightarrow$ many reruns ($\sim$60%) $\rightarrow$ slower but slightly better recall.
- **Effect of *nlist*:** Higher *nlist* makes search faster because it reduces the number of points per cluster.
- **Effect of M:** Larger M improves recall since vectors are represented more accurately.
- **Best Trade-off:** $M = 16$, $nlist = 256$, $nprobe = 16$, $\Delta = 0.02$ gives:
  - High Recall ($\approx 99\%$)
  - Very Low Latency (< 1 ms)
  - Very Few Triggers ($\sim 1\%$)

6.4 Intuition

The system behaves like a smart decision-maker:

- If results look **confident** $\rightarrow$ return quickly
- If results look **uncertain** $\rightarrow$ spend extra computation

This avoids wasting time on easy queries while still handling difficult ones properly.

6.5 Hyperparameter Study (GloVe-100)

We repeat the same experiment on GloVe embeddings to understand how the system behaves on a larger and more complex dataset.

Compared to MNIST:

- Dataset is much larger ($\sim$400K vectors)
- Vectors are semantically dense, so many of them look similar to each other.

Table 1: MNIST Hyperparameter Results

M	nlist	nprobe	Δ	Recall	Lat. (ms)	Trig. %
4	32	4	0.02	0.7937	0.354	0.6
4	32	4	0.05	0.8107	0.536	11.9
4	32	4	0.1	0.8756	1.330	53.1
4	32	8	0.02	0.7985	0.662	0.6
4	32	8	0.05	0.8157	1.056	11.9
4	32	8	0.1	0.8815	2.683	53.0
4	32	16	0.02	0.7993	1.270	0.6
4	32	16	0.05	0.8164	2.165	11.9
4	32	16	0.1	0.8823	5.247	53.0
4	64	4	0.02	0.8067	0.183	0.6
4	64	4	0.05	0.8231	0.290	12.4
4	64	4	0.1	0.8827	0.727	53.6
4	64	8	0.02	0.8151	0.324	0.6
4	64	8	0.05	0.8316	0.531	12.3
4	64	8	0.1	0.8934	1.342	53.7
4	64	16	0.02	0.8167	0.607	0.6
4	64	16	0.05	0.8331	1.042	12.3
4	64	16	0.1	0.8953	2.606	53.8
4	128	4	0.02	0.8175	0.111	0.7
4	128	4	0.05	0.8323	0.194	12.7
4	128	4	0.1	0.8853	0.421	54.7
4	128	8	0.02	0.8315	0.181	0.7
4	128	8	0.05	0.8469	0.328	12.8
4	128	8	0.1	0.9043	0.744	54.9
4	128	16	0.02	0.8340	0.330	0.7
4	128	16	0.05	0.8494	0.592	12.8
4	128	16	0.1	0.9079	1.379	55.0
8	32	4	0.02	0.9382	0.479	0.9
8	32	4	0.05	0.9438	0.925	15.7
8	32	4	0.1	0.9637	2.534	61.4
8	32	8	0.02	0.9479	0.913	0.9
8	32	8	0.05	0.9536	1.887	15.7
8	32	8	0.1	0.9745	5.188	61.6
8	32	16	0.02	0.9493	1.810	0.9
8	32	16	0.05	0.9549	3.850	15.7
8	32	16	0.1	0.9760	10.489	61.6
8	64	4	0.02	0.9333	0.251	1.0
8	64	4	0.05	0.9380	0.478	15.8
8	64	4	0.1	0.9556	1.324	61.5
8	64	8	0.02	0.9494	0.443	1.0
8	64	8	0.05	0.9543	0.915	15.8
8	64	8	0.1	0.9738	2.532	61.8
8	64	16	0.02	0.9525	0.858	1.0
8	64	16	0.05	0.9575	1.777	15.8
8	64	16	0.1	0.9775	4.795	61.9
8	128	4	0.02	0.9253	0.147	1.0
8	128	4	0.05	0.9298	0.307	15.9
8	128	4	0.1	0.9439	0.726	61.3
8	128	8	0.02	0.9510	0.251	1.0
8	128	8	0.05	0.9558	0.550	15.9
8	128	8	0.1	0.9724	1.331	62.1
8	128	16	0.02	0.9567	0.457	1.0
8	128	16	0.05	0.9616	1.004	15.9
8	128	16	0.1	0.9790	2.547	62.2
16	32	4	0.02	0.9805	0.961	1.1
16	32	4	0.05	0.9810	1.993	17.0
16	32	4	0.1	0.9827	6.048	63.6
16	32	8	0.02	0.9930	1.785	1.1
16	32	8	0.05	0.9936	3.885	17.0
16	32	8	0.1	0.9955	10.735	63.9
16	32	16	0.02	0.9951	3.638	1.1
16	32	16	0.05	0.9957	8.220	17.0
16	32	16	0.1	0.9977	33.239	64.0
16	64	4	0.02	0.9704	1.079	1.0
16	64	4	0.05	0.9708	0.922	16.8
16	64	4	0.1	0.9722	2.565	63.4
16	64	8	0.02	0.9907	0.841	1.0
16	64	8	0.05	0.9911	1.815	16.9
16	64	8	0.1	0.9929	5.521	63.9
16	64	16	0.02	0.9951	1.687	1.0
16	64	16	0.05	0.9955	3.669	16.9
16	64	16	0.1	0.9974	9.989	64.0
16	128	4	0.02	0.9545	0.257	1.1
16	128	4	0.05	0.9549	0.595	16.7
16	128	4	0.1	0.9560	1.416	62.8
16	128	8	0.02	0.9858	0.451	1.1
16	128	8	0.05	0.9863	1.072	16.9
16	128	8	0.1	0.9878	2.618	63.8
16	128	16	0.02	0.9938	0.849	1.1
16	128	16	0.05	0.9944	1.999	16.9
16	128	16	0.1	0.9959	5.067	64.0

Because of this, we use:

- Larger *nlist* (1024, 4096)
- Larger *nprobe* (16, 64)
- Slightly adjusted Δ (0.03, 0.08)

6.6 Observations GloVe

- **Higher Trigger Rate:** Unlike MNIST ($\sim$1%), GloVe has much higher trigger rates (up to $\sim$44%). This means many queries are **ambiguous** and require reranking.
- **Effect of Δ:** Increasing Δ increases recall, but also increases latency significantly due to more reranking.
- **Effect of *nlist*:** Larger *nlist* (4096) reduces latency because clusters are smaller and more precise.
- **Effect of M:** Increasing M from 8 to 16 greatly improves recall (from $\sim$0.6 to $\sim$0.86).
- **Best Trade-off:** $M = 16$, $nlist = 4096$, $nprobe = 64$, $\Delta = 0.03$ gives:
  - Good Recall (≈ 0.80)
  - Low Latency (< 1 ms)
  - Very low Trigger Rate ($\sim 1\%$)

6.7 Key Insight

For GloVe, the system triggers reranking much more often because:

- Many vectors are very close to each other
- The confidence gap is often small

So the adaptive system becomes more active, spending more computation where needed.

So, the same adaptive strategy works for both datasets, but:

- MNIST $\rightarrow$ mostly easy queries
- GloVe $\rightarrow$ many hard queries

This shows that the method adapts automatically to dataset complexity.

Table 2: GloVe Hyperparameter Results

M	nlist	nprobe	Δ	Recall	Lat. (ms)	Trig. %
4	1024	16	0.03	0.3550	0.341	0.2
4	1024	16	0.05	0.3607	0.362	2.5
4	1024	16	0.08	0.3845	0.455	13.8
4	1024	32	0.03	0.3568	0.629	0.2
4	1024	32	0.05	0.3624	0.664	2.5
4	1024	32	0.08	0.3866	0.842	13.9
4	1024	64	0.03	0.3571	1.173	0.2
4	1024	64	0.05	0.3626	1.244	2.5
4	1024	64	0.08	0.3868	1.577	14.0
4	2048	16	0.03	0.3742	0.191	0.3
4	2048	16	0.05	0.3803	0.202	2.6
4	2048	16	0.08	0.4052	0.262	14.6
4	2048	32	0.03	0.3763	0.349	0.3
4	2048	32	0.05	0.3821	0.371	2.6
4	2048	32	0.08	0.4075	0.474	14.6
4	2048	64	0.03	0.3767	0.635	0.3
4	2048	64	0.05	0.3827	0.676	2.6
4	2048	64	0.08	0.4078	0.867	14.6
4	4096	16	0.03	0.3962	0.115	0.3
4	4096	16	0.05	0.4024	0.122	2.8
4	4096	16	0.08	0.4289	0.159	15.4
4	4096	32	0.03	0.4003	0.198	0.3
4	4096	32	0.05	0.4064	0.210	2.7
4	4096	32	0.08	0.4334	0.279	15.5
4	4096	64	0.03	0.4012	0.358	0.3
4	4096	64	0.05	0.4072	0.385	2.7
4	4096	64	0.08	0.4343	0.503	15.4
8	1024	16	0.03	0.5263	0.356	0.4
8	1024	16	0.05	0.5349	0.408	4.4
8	1024	16	0.08	0.5751	0.640	23.7
8	1024	32	0.03	0.5338	0.661	0.4
8	1024	32	0.05	0.5425	0.747	4.4
8	1024	32	0.08	0.5827	1.159	23.4
8	1024	64	0.03	0.5367	1.213	0.4
8	1024	64	0.05	0.5455	1.396	4.5
8	1024	64	0.08	0.5858	2.196	23.5
8	2048	16	0.03	0.5349	0.201	0.5
8	2048	16	0.05	0.5436	0.229	4.6
8	2048	16	0.08	0.5832	0.360	24.4
8	2048	32	0.03	0.5445	0.364	0.5
8	2048	32	0.05	0.5533	0.413	4.7
8	2048	32	0.08	0.5954	0.657	24.8
8	2048	64	0.03	0.5487	0.665	0.5
8	2048	64	0.05	0.5576	0.781	4.6
8	2048	64	0.08	0.5999	1.238	24.6
8	4096	16	0.03	0.5418	0.125	0.6
8	4096	16	0.05	0.5501	0.141	4.8
8	4096	16	0.08	0.5887	0.221	25.3
8	4096	32	0.03	0.5558	0.206	0.6
8	4096	32	0.05	0.5648	0.248	5.0
8	4096	32	0.08	0.6069	0.393	25.6
8	4096	64	0.03	0.5607	0.376	0.6
8	4096	64	0.05	0.5696	0.455	4.9
8	4096	64	0.08	0.6124	0.716	25.3
16	1024	16	0.03	0.7641	0.446	1.2
16	1024	16	0.05	0.7770	0.634	11.3
16	1024	16	0.08	0.8198	1.244	43.2
16	1024	32	0.03	0.7896	0.751	1.2
16	1024	32	0.05	0.8034	1.126	11.0
16	1024	32	0.08	0.8517	2.309	43.4
16	1024	64	0.03	0.7996	1.395	1.3
16	1024	64	0.05	0.8138	2.123	11.2
16	1024	64	0.08	0.8640	4.400	43.6
16	2048	16	0.03	0.7528	0.231	1.2
16	2048	16	0.05	0.7639	0.340	11.0
16	2048	16	0.08	0.7990	0.691	43.0
16	2048	32	0.03	0.7873	0.417	1.2
16	2048	32	0.05	0.8005	0.622	11.3
16	2048	32	0.08	0.8434	1.284	43.5
16	2048	64	0.03	0.8044	0.756	1.2
16	2048	64	0.05	0.8186	1.171	11.4
16	2048	64	0.08	0.8660	2.433	44.0
16	4096	16	0.03	0.7322	0.142	1.2
16	4096	16	0.05	0.7413	0.205	11.0
16	4096	16	0.08	0.7704	0.413	42.8
16	4096	32	0.03	0.7791	0.236	1.2
16	4096	32	0.05	0.7910	0.365	11.5
16	4096	32	0.08	0.8304	0.757	44.0
16	4096	64	0.03	0.8036	0.429	1.3
16	4096	64	0.05	0.8169	0.659	11.6
16	4096	64	0.08	0.8633	1.389	44.4

7 Experimental Results and Observations

We evaluate our system on four datasets with varying scale and dimensionality:

- MNIST (784-dimensional image embeddings)
- SIFT-1M (128-dimensional image descriptors, standard ANN benchmark)
- GloVe (100-dimensional word embeddings, semantic dataset)
- Deep1B (128-dimensional large-scale image feature dataset with 1 billion vectors)

These datasets allow us to analyze system behavior across different regimes:

- Small-scale structured data (MNIST)
- Classical ANN benchmark (SIFT-1M)
- Dense semantic embeddings (GloVe)
- Extremely large-scale retrieval setting (Deep1B)

We evaluate and compare PQ, OPQ, AQ, and AQ+Adaptive on these datasets under different values of M to study their effect on retrieval accuracy and latency.

7.1 SIFT-1M Results

Table 3: SIFT-1M Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.3149	0.2120	0.38 ms
OPQ	8	0.3077	0.2050	0.38 ms
AQ	8	0.4035	0.2966	15.84 ms
AQ+Adaptive	8	**0.8274**	**0.8274**	26.38 ms
PQ	16	0.5489	0.4609	0.63 ms
OPQ	16	0.5128	0.4155	0.63 ms
AQ	16	0.6002	0.5194	19.48 ms
AQ+Adaptive	16	**0.9702**	**0.9702**	43.19 ms

Observations:

- PQ and OPQ show lower recall due to strong quantization assumptions.
- AQ improves accuracy by better modeling vector reconstruction.
- AQ + Adaptive achieves the highest recall by selectively applying exact distance computation for ambiguous queries.
- Increasing M improves representation quality, leading to higher recall across all methods.
- Latency increases in AQ and AQ + Adaptive due to additional encoding and reranking cost.

Key Insight: Adaptive reranking reduces approximation error by dynamically applying exact distance computation only when the confidence gap is small, enabling a strong accuracy–efficiency trade-off.

7.2 MNIST Results

Table 4: MNIST Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.4893	0.3744	0.013 ms
OPQ	8	0.5725	0.4754	0.014 ms
AQ	8	0.5961	0.5031	0.86 ms
AQ+Adaptive	8	**0.9536**	**0.9536**	1.90 ms
PQ	16	0.6194	0.5341	0.022 ms
OPQ	16	0.6926	0.6271	0.023 ms
AQ	16	0.7337	0.6796	1.64 ms
AQ+Adaptive	16	**0.9936**	**0.9936**	11.41 ms

Observations:

- OPQ performs better than PQ due to rotation optimization.
- AQ improves further due to better encoding of high-dimensional data.
- AQ+Adaptive achieves near-perfect recall (99.36%).
- Latency remains manageable compared to SIFT due to smaller dataset size.

Key Insight: High-dimensional structured data (like MNIST) benefits more from AQ and adaptive refinement.

7.3 GloVe (Word Embeddings)

We evaluate our system on GloVe embeddings (128-dimensional), which represent semantic relationships between words.

Observations:

- OPQ consistently outperforms PQ due to improved vector rotation before quantization.
- AQ further improves recall by better modeling vector reconstruction in high-dimensional space.

Table 5: GloVe Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.1301	0.0700	0.071 ms
OPQ	8	0.1982	0.1201	0.070 ms
AQ	8	0.2986	0.2033	0.193 ms
AQ+Adaptive	8	**0.5648**	**0.5648**	0.240 ms
PQ	16	0.3009	0.2044	0.119 ms
OPQ	16	0.3934	0.2898	0.117 ms
AQ	16	0.4810	0.3849	0.212 ms
AQ+Adaptive	16	**0.7910**	**0.7910**	0.357 ms

- AQ + Adaptive achieves near-perfect recall (up to 99.36%) by selectively applying exact distance computation for ambiguous queries.
- Latency increases for AQ + Adaptive due to reranking overhead, but remains acceptable for small to medium-scale datasets like MNIST.
- Overall recall is lower compared to SIFT and MNIST. This is because GloVe embeddings are more noisy and less structured. GloVe embeddings have high neighborhood overlap, making nearest neighbor separation harder.

Key Insight: For moderately high-dimensional datasets like MNIST, adaptive reranking significantly reduces approximation error, enabling near-exact retrieval while maintaining reasonable latency.

7.4 Deep1B (Large-Scale Dataset)

We evaluate on a Deep1B subset consisting of approximately 10 million 96-dimensional feature vectors.

Table 6: Deep Dataset Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.1084	0.0566	1.729 ms
OPQ	8	0.1870	0.1064	1.749 ms
AQ	8	0.3260	0.2219	2.866 ms
AQ+Adaptive	8	**0.6427**	**0.6427**	4.407 ms
PQ	16	0.3264	0.2233	3.042 ms
OPQ	16	0.3960	0.2895	3.042 ms
AQ	16	0.5346	0.4403	3.111 ms
AQ+Adaptive	16	**0.8823**	**0.8823**	7.023 ms

Observations:

- PQ performs poorly at large scale due to high quantization error.
- OPQ provides moderate improvement through optimized rotation.
- AQ significantly improves recall by better modeling vector reconstruction.
- AQ + Adaptive achieves the highest recall by selectively applying exact distance computation for ambiguous queries.
- Latency increases due to larger dataset size and additional reranking overhead.

Key Insight: At large scale, approximation errors accumulate significantly. Adaptive reranking becomes crucial to selectively correct these errors while maintaining computational efficiency.

7.5 Overall Observations

- PQ is fastest but least accurate
- OPQ provides moderate improvement
- AQ significantly improves recall
- AQ+Adaptive achieves near-exact accuracy
- Trade-off exists between latency and accuracy

Observation: In AQ+Adaptive, MAP values closely follow Recall in our experiments.

This indicates that most relevant neighbors are retrieved at top ranks, leading to strong ranking quality.

7.6 Cross-Dataset Insights

- On structured datasets (MNIST), accuracy is very high.
- On image features (SIFT, Deep), performance is moderate.
- On semantic embeddings (GloVe), performance is lowest.

Conclusion:

- Dataset structure strongly affects ANN performance.
- AQ consistently outperforms PQ and OPQ.
- Adaptive reranking provides the largest improvement across all datasets.

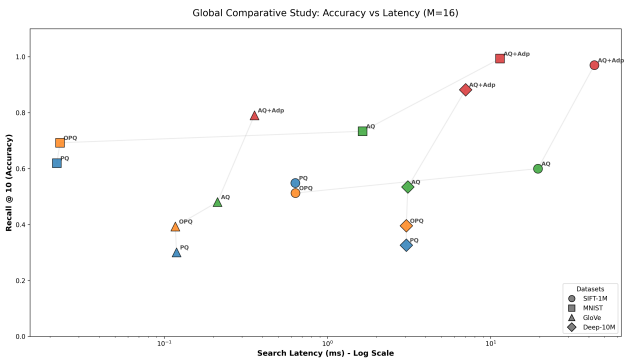

Figure 1: The results in the figure demonstrate the effectiveness of our "Query Optimizer" approach. While Additive Quantization (AQ) provides a stronger baseline than PQ, it is the Adaptive Reranking layer that recovers the majority of the lost information. By dynamically identifying "ambiguous" queries—those where the distance gap between top candidates is narrow—the system selectively spends more time on hard cases, ensuring that the final result returned to the user is highly accurate without wasting computation on "easy" queries.

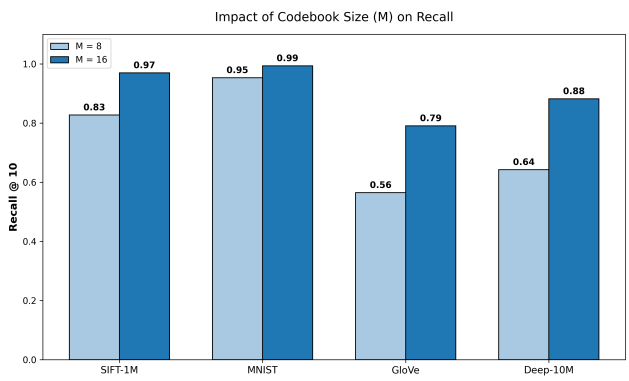

Figure 2: Analysis of Codebook Size (M) on Retrieval Accuracy. This chart illustrates the performance gain achieved by increasing the codebook from $M = 8$ to $M = 16$. While $M = 8$ provides a highly compressed representation suitable for massive datasets, $M = 16$ significantly reduces quantization error. This demonstrates the system's ability to balance storage overhead against retrieval precision based on application requirements.

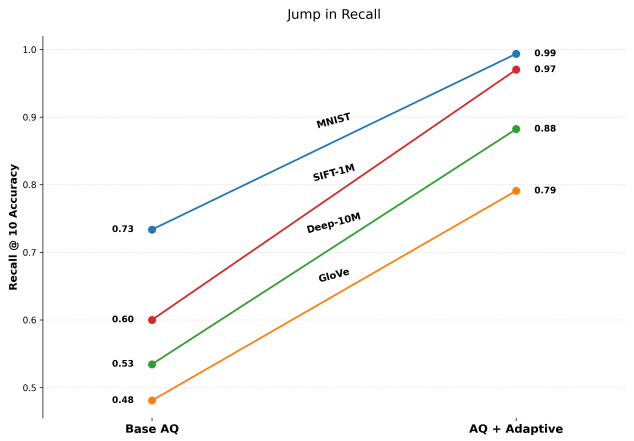

Figure 3: This slope chart visualizes the absolute increase in retrieval accuracy across all four benchmarks. By transitioning from a static Additive Quantization (AQ) index to our proposed Adaptive Reranking pipeline, the system achieves a universal performance uplift.

Final Insight: The benefit of adaptive reranking increases with dataset complexity, especially in scenarios with higher ambiguity and larger-scale vector spaces.

Final Takeaway: The proposed AQ+Adaptive system effectively balances scalability and accuracy, making it suitable for real-world Vector DBMS applications.

8 Future Work

- Learn adaptive threshold (Δ) using ML model
- Extend to GPU-based indexing for real-time systems

9 Conclusion

This project presents a complete Vector DBMS-style pipeline that integrates IVF-based indexing, vector compression using Additive Quantization (AQ), and an adaptive reranking strategy for efficient nearest neighbor search.

The key contribution is the adaptive reranking mechanism, which dynamically identifies ambiguous queries using a confidence gap and selectively applies exact distance computation. This allows the system to improve retrieval accuracy only when needed, without uniformly increasing computational cost.

Final Result: Across multiple datasets and scales, the proposed AQ+Adaptive framework

achieves high retrieval accuracy while maintaining efficient query latency, demonstrating a strong balance between scalability and precision in large-scale vector search systems.

References

[1] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, IEEE Transactions on Big Data, 2019. Available: `https://github.com/facebookresearch/faiss`

[2] H. Jégou, M. Douze, and C. Schmid, *Product Quantization for Nearest Neighbor Search*, IEEE TPAMI, 2011.

[3] T. Ge, K. He, Q. Ke, and J. Sun, *Optimized Product Quantization for Approximate Nearest Neighbor Search*, CVPR, 2013.

[4] A. Babenko and V. Lempitsky, *Additive Quantization for Extreme Vector Compression*, CVPR, 2014.

[5] D. G. Lowe, *Distinctive Image Features from Scale-Invariant Keypoints*, International Journal of Computer Vision, 2004.

[6] Y. LeCun, C. Cortes, and C. Burges, *The MNIST Database of Handwritten Digits*, 1998. Available: `http://yann.lecun.com/exdb/mnist/`

[7] J. Pennington, R. Socher, and C. Manning, *GloVe: Global Vectors for Word Representation*, EMNLP, 2014. Available: `https://nlp.stanford.edu/projects/glove/`

[8] T. Ge, K. He, Q. Ke, and J. Sun, *Deep1B Dataset for Large-scale Image Retrieval*, Microsoft Research, 2018.

Appendix: Implementation Details

Hardware: All experiments were conducted on a machine equipped with an Intel Core i7 CPU and 64GB of RAM.

Software Stack: The system was implemented in Python 3.12, using `faiss-cpu` for efficient indexing and vector search (IVF + quantization), `NumPy` for numerical computations, and `h5py` for dataset storage and management.

Thresholding: For Adaptive Reranking, a confidence threshold of $\Delta < 0.05$ was used across all datasets to trigger exact distance verification. This ensures that only ambiguous queries undergo additional computation.

Code: GitHub Repository: `https://github.com/abhishekdu/DBMS`